PROFESSIONAL TRAINING MATERIAL • 2026 EDITION

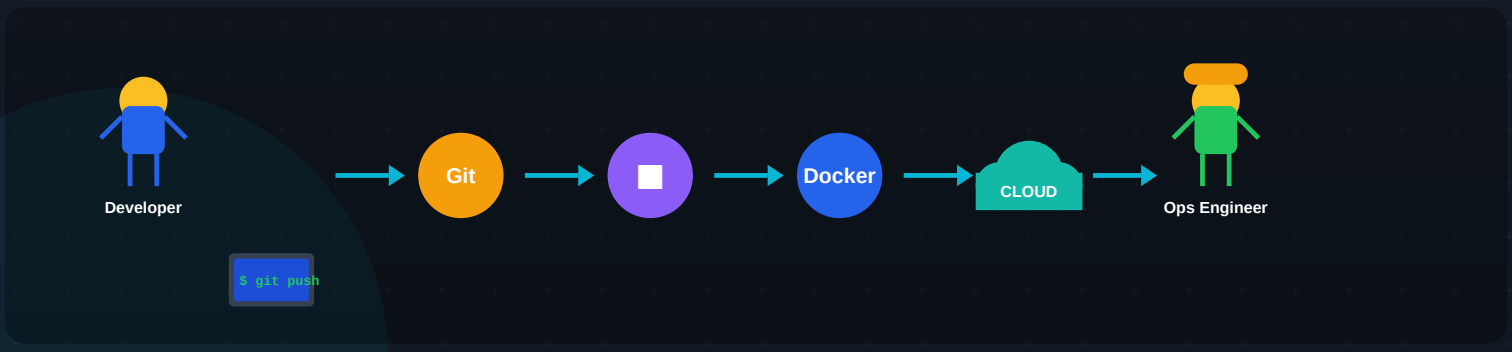
DevOps & Cloud — 90-Day Learning Journey

DAY 01 • MODULE 1

Introduction to DevOps, Cloud & SRE

Core concepts, culture, lifecycle, and career foundations

14 Chapters | Bullet-First | Try It Tasks | Mini Quizzes



■ Beginner | ■ ~90 min read

Aryashree Pritikrishna

github.com/aryashreep/learn-devops-in-90-days

✦ Follow along: 90-day hands-on DevOps learning journey ✦

Table of Contents — Day 01

Ch 13	Real-World DevOps & Cloud Workflow	■ Intermediate ■10m
Ch 14	Career Path Overview	■ Beginner ■9m

CHAPTER 01 ■ Beginner ■ 8 min read 1/14

Software Development & IT Operations

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

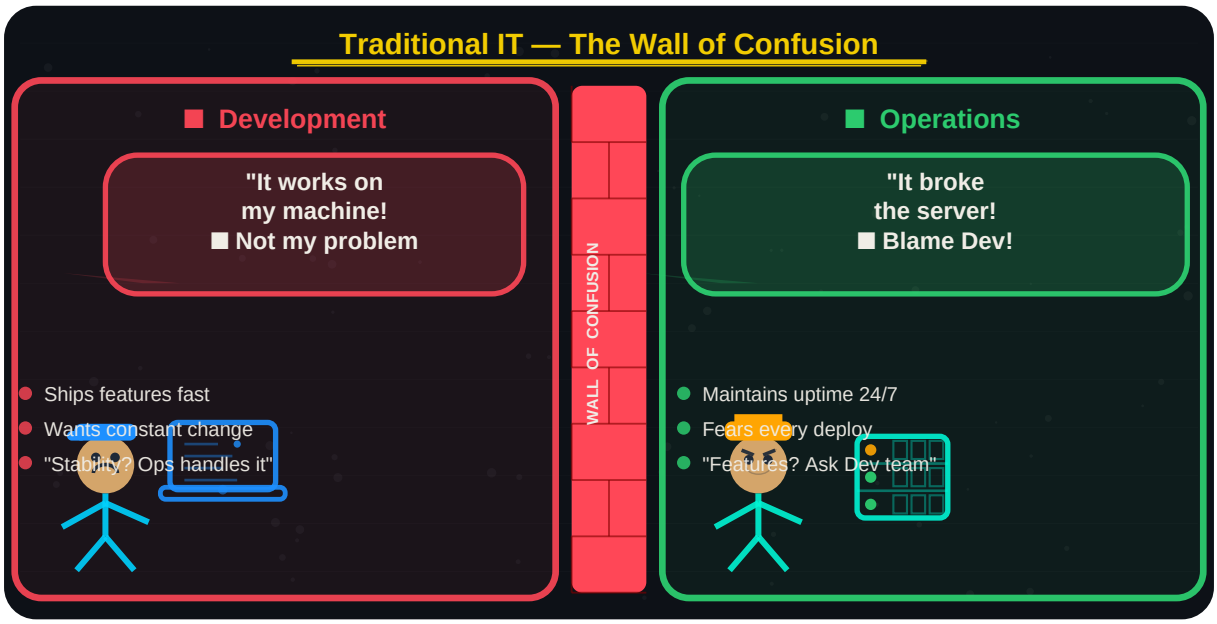
- ❖ Explain the difference between Development and Operations in plain English
- ❖ Define SLA and MTTR and explain why they matter
- ❖ Describe why Dev and Ops traditionally had conflicting goals
- ❖ Give a real-world example of Dev and Ops needing to work together

■ ANALOGY

Software company = restaurant → Dev are the chefs, Ops are the floor managers

◆ DID YOU KNOW? ◆

Dev	Ops	Both
team builds & ships features	team runs & protects systems	essential for delivering value



Traditional IT: Dev and Ops separated by the Wall of Confusion — causing blame, delays, and outages

Key Concepts

- **Development (Dev):** team responsible for building software
  - → writes application code, designs new features
  - → fixes bugs, improves performance
  - → focuses on: speed of delivery and new functionality
- **Operations (Ops):** team responsible for running software

→ → manages servers, networks, security, cloud infrastructure

→ → ensures 24/7 uptime and system reliability

→ → focuses on: stability, performance, and zero downtime

- **SLA (Service Level Agreement):** formal uptime contract
  - → → defines minimum acceptable uptime (e.g., 99.9% = max 8.7 hrs downtime/year)
  - → → Ops must meet SLA even as Dev pushes constant changes
  - → → breaching an SLA often has financial penalties
- **MTTR (Mean Time to Recovery):** speed of incident response
  - → → measures how fast a team restores service after an outage
  - → → DevOps goal: reduce MTTR from hours → minutes
  - → → lower MTTR = less customer impact = better reputation

■ Traditional IT	■ DevOps Way
• Dev throws code "over the wall"	• Dev and Ops plan together from day one
• Ops learns about changes on release day	• Shared dashboards and on-call rotations
• Blame game when things break	• Blameless post-mortems — fix systems not people
• Separate tools, no shared visibility	• Unified toolchain with full transparency

■ **REAL-WORLD EXAMPLE** — HDFC Bank

Situation:

- Dev team built a cheque-deposit-by-camera feature; Ops managed image-processing servers

What they did:

- Coordinated deployment windows between Dev and Ops teams
- Ops pre-scaled servers before the feature release
- Dev wrote automated tests for server capacity thresholds

Result:

- Zero downtime during feature launch affecting 40M+ users

■ **COMMON MISCONCEPTIONS**

■ **Myth:**

Ops are just customer support who answer phone calls

■ **Reality:**

Ops engineers are highly technical — they manage cloud infrastructure, networking, security, and complex distributed systems

■ **Myth:**

Dev is more important than Ops — Ops just maintains things

■ **Reality:**

Both are equally critical. Dev without Ops = code with nowhere to run. Ops without Dev = servers with no software to deliver

■ TRY IT YOURSELF

1. Draw a diagram of your current or imagined company — label who is "Dev" and who is "Ops"
2. Calculate: what does 99.9% uptime SLA mean in minutes of allowed downtime per month?
3. Find and read one public incident post-mortem (try: github.blog/tag/incident or status.atlassian.com)

✓ QUICK SUMMARY

- **Development (Dev):** builds software — focused on speed and new features
- **Operations (Ops):** runs software — focused on stability and reliability
- **SLA:** formal uptime contract — e.g. 99.9% uptime = max 8.7 hrs downtime/year
- **MTTR:** how fast you recover from outages — DevOps targets minutes not hours
- **Conflict:** Dev wants change fast; Ops wants stability — DevOps resolves this tension
- **Both teams** are essential — neither succeeds without the other

■ MINI QUIZ — Test Your Understanding

Q1. What does MTTR stand for?

a) Mean Time To Release
b) Mean Time To Recovery
c) Maximum Time To Resolve
d) Mean Team Task Rate

✓ Answer: b — Mean Time to Recovery — measures how fast a team restores service after failure

Q2. A bank releases a new feature and the servers crash. Whose fault is this?

a) Only Dev — they wrote bad code
b) Only Ops — they misconfigured servers
c) Both teams — they failed to coordinate
d) Neither — outages are inevitable

✓ Answer: c — DevOps teaches shared responsibility — both teams own the outcome together

Q3. An SLA of 99.99% allows how many hours of downtime per year?

a) 8.76 hours
b) 52.6 minutes
c) 1 hour
d) 24 hours

✓ Answer: b — 99.99% uptime = 0.01% downtime = 52.6 minutes max per year

CHAPTER 02 ■ Beginner ■ 7 min read 2/14

Problems with Traditional IT

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ❖ Explain the "Wall of Confusion" and why it caused project failures
- ❖ Describe why Waterfall model is unsuitable for modern software delivery
- ❖ Identify at least 3 bottlenecks common in traditional IT organisations
- ❖ Relate traditional IT problems to real business impact (lost revenue, outages)

■ ANALOGY

Traditional IT = chefs and waiters who never talk → food arrives cold, wrong, and late

◆ DID YOU KNOW? ◆

6-18M	$4.5M	70%
typical Waterfall release cycle	avg cost of a major e-commerce outage	IT projects fail due to poor collaboration

Why Traditional IT Failed

- **The Wall of Confusion:** invisible barrier between Dev and Ops
  - → → Dev: "It works on my machine!" → Ops: "It broke our servers!"
  - → → Neither team understands the other's tools, constraints, or pressures
  - → → Result: blame games, delayed fixes, and repeated outages
- **Waterfall Model:** slow, sequential, high-risk releases
  - → → Phases in strict order: Requirements → Design → Code → Test → Deploy
  - → → Each phase must fully complete before the next begins
  - → → Testing happens LAST — bugs found after months of development
  - → → Release cycles: 6–18 months → dangerously slow for modern business
- **Silos:** teams working in total isolation
  - → → Dev team has no visibility into production infrastructure
  - → → Ops team has no visibility into upcoming code changes
  - → → Separate tools, separate dashboards, separate on-call processes
- **Manual Bottlenecks:** humans doing what machines should do
  - → → Manual server provisioning: "Click through AWS console for 3 days"
  - → → Manual testing: "QA team runs 500 test cases by hand over 2 weeks"
  - → → Manual deployments: "SSH into each server and restart the service"
  - → → Result: slow, error-prone, inconsistent, and impossible to scale
- **Integration Hell:** the cost of merging months of parallel work

→ → Developers work in isolation for weeks or months

→ → When code is merged → massive conflicts, broken functionality

→ → Fixing merge conflicts can take as long as writing the original code

■ REAL-WORLD EXAMPLE

Amazon (pre-DevOps)

Situation:

- Teams worked in silos; deployments happened every few months with huge risk of failure

What they did:

- Adopted DevOps culture with shared ownership across Dev and Ops
- Moved from Waterfall to continuous deployment model
- Built internal tools to automate deployments and testing

Result:

- Deployment frequency: from every few months → every 11.7 seconds

■ COMMON MISCONCEPTIONS

■ Myth:

"It works on my machine" means the developer did a good job

■ Reality:

It means the development environment differs from production — DevOps eliminates this with consistent, code-defined environments

■ Myth:

Waterfall is safer because you test everything at the end

■ Reality:

Testing last is the riskiest approach — bugs found late are 100x more expensive to fix than bugs found early

■ TRY IT YOURSELF

1. List 3 manual tasks you do weekly at work or in a project — estimate time spent on each
2. Read about the 2021 Facebook 6-hour outage (search "Facebook BGP outage 2021") — identify which silo failures caused it
3. Time yourself: how long does it take to set up a new development environment manually vs using a script?

✓ QUICK SUMMARY

- **Wall of Confusion:** cultural/technical barrier between Dev and Ops — root cause of most outages
- **Waterfall:** sequential model with 6-18 month cycles — unsuitable for modern competitive markets
- **Silos:** teams in isolation → no shared knowledge → repeated mistakes
- **Bottlenecks:** manual testing, provisioning, deployment → slow, error-prone delivery
- **Integration Hell:** merging months of separate work = weeks of painful conflict resolution
- **DevOps solution:** break silos, automate bottlenecks, release small and often

■ MINI QUIZ — Test Your Understanding

Q1. What is "Integration Hell" in traditional software development?

a) Server overheating during deployment

b) Painful process of merging months of separate code branches

c) When the internet goes down during a release

d) When QA rejects all test cases

✓ Answer: b — Integration Hell occurs when parallel development branches diverge so much that merging them causes massive conflicts

Q2. A company releases software every 6 months using Waterfall. Netflix releases hundreds of times daily. What is the key difference?

a) Netflix has more developers

b) Netflix uses better programming languages

c) Netflix uses DevOps with CI/CD automation

d) Netflix has fewer features to release

✓ Answer: c — DevOps with CI/CD enables continuous, automated releases — Waterfall creates bottlenecks that force large, infrequent releases

Q3. Which of these is NOT a problem with siloed organisations?

a) Dev does not understand Ops constraints

b) Teams use the same deployment tools

c) Blame games when production breaks

d) Manual processes that cannot scale

✓ Answer: b — Siloed orgs typically use DIFFERENT tools with no integration — shared tooling is a DevOps characteristic

CHAPTER 03 ■ Beginner ■ 8 min read 3/14

What is DevOps?

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

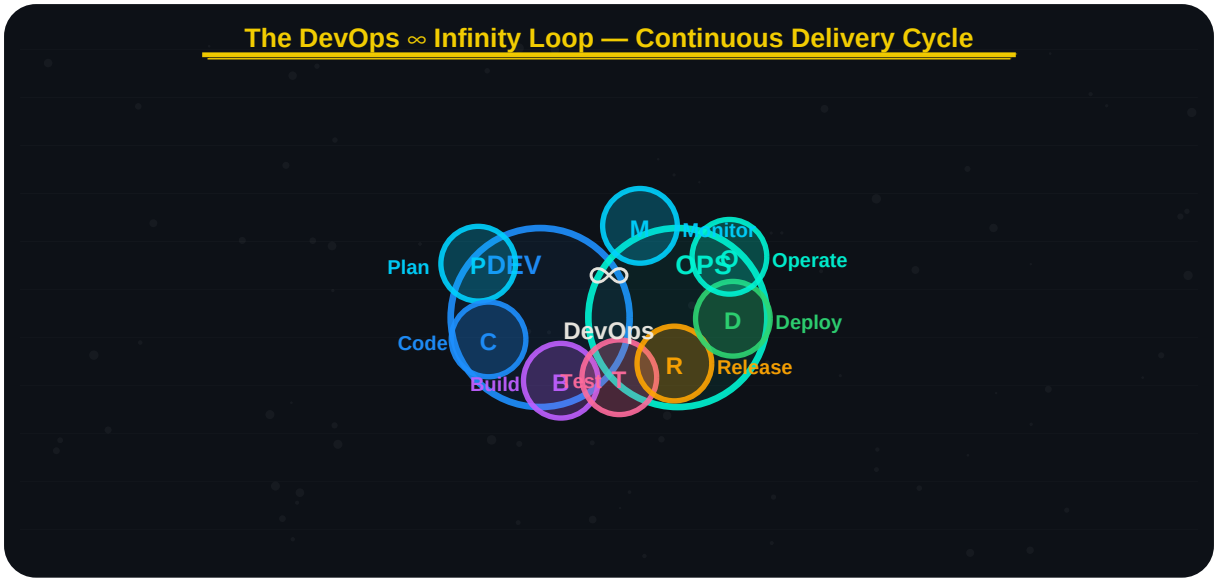
- ❖ Define DevOps accurately — culture, methodology, and shared ownership
- ❖ Explain why DevOps is NOT a job title, tool, or department
- ❖ Describe the mindset shift from "you build it, I run it" to "you build it, you run it"
- ❖ Cite real metrics showing DevOps business impact (Amazon, Netflix)

■ ANALOGY

DevOps = removing the wall between chefs and waiters → everyone owns the restaurant's success

◆ DID YOU KNOW? ◆

11.7s	100s	50%
Amazon's average deployment cycle	Netflix daily deployments	faster time-to-market with DevOps



The DevOps Infinity Loop — 8 stages of continuous, iterative software delivery

What DevOps Actually Is

- **DevOps is a culture:** shared values, behaviours, and practices
  - → → trust between Dev and Ops — both own the product outcome
  - → → psychological safety — freedom to try, fail, and learn
  - → → blameless mindset — fix systems, not blame people
- **DevOps is a methodology:** a structured approach to delivery
  - → → continuous planning, building, testing, deploying, and learning

→ → small, frequent releases instead of large, infrequent ones

→ → automation at every step — humans focus on creative work

- **DevOps is shared ownership:** "you build it, you run it"
  → → developers are on-call for services they wrote
  → → Ops engineers contribute to architecture decisions
  → → everyone is responsible from idea to production to monitoring

What DevOps Is NOT

- **Not a job title:** "DevOps Engineer" is a role, but DevOps itself is an org-wide culture
- **Not a tool:** buying Jenkins or Docker alone does NOT give you DevOps
- **Not a team:** creating a "DevOps team" that does deployments for everyone defeats the purpose
- **Not just automation:** you can automate bad processes — DevOps requires improving them first

■ Traditional IT	■ DevOps Mindset
• "You build it, I run it"	• "You build it, you run it"
• Dev and Ops on separate floors	• Dev and Ops share standups + dashboards
• Ops learns about release on deploy day	• Ops co-designs architecture with Dev
• Fear of deploying on Fridays	• Deploy any day — automated rollback if issues

■ **REAL-WORLD EXAMPLE** — Amazon

Situation:

- Releasing software every few months with high risk — entire company slowed by deployment fear

What they did:

- Adopted "two-pizza team" model — small, autonomous teams own their services end-to-end
- Built internal CI/CD tooling (later became AWS CodePipeline)
- Gave developers full ownership: build, deploy, monitor, and on-call for their own services

Result:

- Deployment cycle: from months → every 11.7 seconds
- 23,000+ deployments per day across the entire organisation
- Foundation for AWS — a $90B/year business born from DevOps tooling

■ COMMON MISCONCEPTIONS

■ Myth:

DevOps means developers do all the operations work themselves

■ Reality:

DevOps means Dev and Ops collaborate with shared responsibility — specialised skills still exist, silos do not

■ Myth:

One person can "implement DevOps" for a company

■ Reality:

DevOps requires the entire organisation to change culture — tools, processes, and leadership alignment all matter

■ TRY IT YOURSELF

1. Watch "10+ Deploys Per Day" on YouTube (the original DevOps talk by Flickr engineers, 2009)
2. Calculate: if Amazon deploys every 11.7 seconds, how many deployments happen in one working day (8 hours)?
3. Identify one process in your current work or studies that would benefit most from "continuous improvement" — write down how you would measure it

✓ QUICK SUMMARY

- **DevOps = culture + methodology + shared ownership** — not a tool or job title
- **Mindset shift:** "You build it, you run it" → everyone owns the outcome
- **Goal:** deliver value faster, more reliably, and with fewer bugs
- **Amazon proof:** 11.7-second deployment cycles → zero downtime at massive scale
- **Not just automation:** culture must change first — then automate improved processes
- **Whole org must adopt:** DevOps cannot succeed in one team while others resist it

■ MINI QUIZ — Test Your Understanding

Q1. Which of these BEST describes DevOps?

a) A specific tool like Jenkins or Docker

b) A job title for someone who does deployments

c) A culture and methodology combining Dev and Ops with shared ownership

d) A replacement for the Operations team

✓ Answer: c — DevOps is fundamentally a cultural and methodological shift — tools and titles are secondary

Q2. A company buys Kubernetes and Jenkins but keeps Dev and Ops in separate departments with no collaboration. Is this DevOps?

a) Yes — they have the right tools

b) No — this is Cargo Cult DevOps without the culture

c) Partially — tools are 50% of DevOps

d) Yes — Kubernetes alone enables DevOps

✓ Answer: b — Cargo Cult DevOps: copying the tools without the culture always fails — culture is the foundation

Q3. Amazon deploys every 11.7 seconds. How many deployments is that per hour?

a) About 100

b) About 200

c) About 308

d) About 500

✓ Answer: c — 3600 seconds ÷ 11.7 = ~307.7 deployments per hour across Amazon

CHAPTER 04 ■ Beginner ■ 9 min read 4/14

DevOps Culture & Principles — C.A.L.M.S.

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ❖ Explain each letter of C.A.L.M.S. and give a practical example for each
- ❖ Describe what a blameless post-mortem is and why it builds stronger systems
- ❖ Distinguish between genuine DevOps culture and "Cargo Cult DevOps"
- ❖ Identify the most important pillar of CALMS and explain why

■ ANALOGY

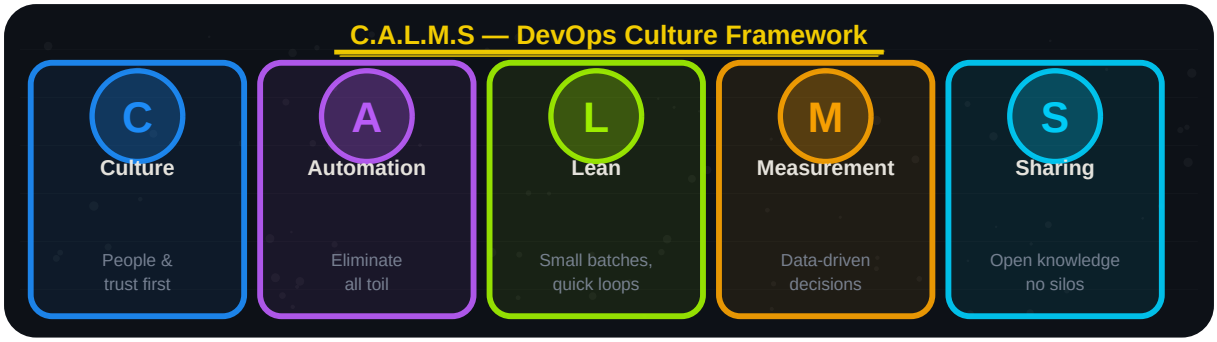
CALMS culture = sports team → trust, strategy, measurement, learning from every game

◆ DID YOU KNOW? ◆

Culture must come before tools — always

Blameless post-mortems build more resilient systems

Toil = manual repetitive work → automate it all



C.A.L.M.S. — the five pillars of sustainable DevOps culture

C.A.L.M.S. — Each Pillar Explained

- **C — Culture:** People, trust, and psychological safety
  - → → people > process > tools — always in that order
  - → → psychological safety: freedom to try, fail, and improve without fear
  - → → blameless post-mortems: incidents are system problems, not people problems
  - → → practical example: Etsy does not fire engineers for causing outages — they improve systems
- **A — Automation:** Eliminate toil — repetitive manual work
  - → → toil = work that is manual, repetitive, automatable, and scales with service growth
  - → → automate: testing, deployments, server provisioning, alerting, backups
  - → → rule of thumb: if you do it more than twice manually → automate it
  - → → practical example: replacing "ssh into 50 servers and restart nginx" with one Ansible command
- **L — Lean:** Minimise waste and maximise flow

→ → small batch sizes: release small changes frequently instead of huge releases rarely

→ → fast feedback loops: learn in minutes, not months

→ → eliminate waste: unnecessary approvals, waiting, rework, over-engineering

→ → practical example: feature flags let you ship incomplete features safely and enable them gradually

- **M — Measurement:** Data-driven decisions — measure everything
  - → → deployment frequency: how often do you release? (goal: multiple times daily)
  - → → lead time: how long from code commit to production? (goal: under 1 hour)
  - → → MTTR: how fast do you recover from incidents? (goal: under 30 minutes)
  - → → change failure rate: % of releases causing incidents (goal: under 5%)
- **S — Sharing:** Open knowledge, no hoarding, no silos
  - → → share post-mortems openly — the whole organisation learns from one team's incident
  - → → open source internal tools — if your build tool is useful, share it
  - → → cross-team documentation: runbooks, architecture diagrams, decision logs
  - → → practical example: Google publishes entire SRE book for free — radically open sharing

Pillar	Key Practice	Anti-Pattern (what to avoid)
C — Culture	Blameless post-mortems after incidents	Firing engineers who cause outages
A — Automation	Scripts, pipelines, IaC for all repetitive tasks	Doing the same manual step 100 times
L — Lean	Small PRs, feature flags, frequent releases	Releasing once every 6 months
M — Measurement	DORA metrics: frequency, lead time, MTTR, CFR	Guessing without data
S — Sharing	Open wikis, shared runbooks, public post-mortems	"That's my team's knowledge, not yours"

■ REAL-WORLD EXAMPLE — Etsy

Situation:

- Engineer accidentally deleted production database causing a major outage

What they did:

- Ran a blameless post-mortem — no punishment for the engineer
- Entire team analysed root cause: "why was deleting production data so easy?"
- Added deletion confirmation steps, automated backups, and access controls

Result:

- The engineer stayed at Etsy and became a senior engineer
- The database was protected by 3 additional safeguards
- Culture of openness led to more engineers reporting near-misses early

■ COMMON MISCONCEPTIONS

■ Myth:

Buying DevOps tools (Jenkins, Docker, Kubernetes) means you have DevOps

■ Reality:

Cargo Cult DevOps — copying the tools without the culture always fails. Culture must come first, then process, then tools

■ Myth:

Automation means letting machines do everything — humans are not needed

■ Reality:

Automation eliminates toil so humans can focus on creative, high-value problem-solving — not replacing humans

■ TRY IT YOURSELF

1. Read Google's SRE blameless post-mortem chapter — it is free at sre.google/sre-book/postmortem-culture
2. Identify one "toil" task in your current workflow — write a 5-line bash script to partially automate it
3. Look up "DORA metrics" — write down what each of the 4 metrics measures and what "elite performer" numbers look like

✓ QUICK SUMMARY

- **Culture first:** people > process > tools — always in that order
- **C.A.L.M.S.:** Culture, Automation, Lean, Measurement, Sharing — the 5 DevOps pillars
- **Blameless post-mortems:** incidents reveal system weaknesses — fix the system, not the person
- **Toil:** manual repetitive work — automate anything done more than twice
- **DORA metrics:** measure deployment frequency, lead time, MTTR, and change failure rate
- **Cargo Cult DevOps:** tools without culture = expensive failure — mindset must change first

■ MINI QUIZ — Test Your Understanding

Q1. What does the "L" in C.A.L.M.S. stand for?

a) Learning

b) Leadership

c) Lean

d) Logging

✓ Answer: c — Lean — minimise waste, reduce batch sizes, enable fast feedback loops

Q2. An engineer causes a production outage. The DevOps approach is to:

a) Fire the engineer immediately

b) Document the failure and run a blameless post-mortem

c) Blame the engineer publicly as a warning to others

d) Ignore it and hope it does not happen again

✓ Answer: b — Blameless post-mortems focus on systemic improvements — blame culture suppresses transparency and makes systems less safe

Q3. Which of these is a DORA "elite performer" metric?

a) Deploy once per month

b) MTTR of 1 week

c) Change failure rate of 30%

d) Multiple deployments per day

✓ Answer: d — Elite DORA performers deploy multiple times per day with MTTR under 1 hour and change failure rate under 5%

CHAPTER 05 ■ Beginner ■ 8 min read 5/14

The DevOps Lifecycle — 8 Stages

■ **LEARNING OBJECTIVES**

By the end of this chapter you will be able to:

- ❖ Name all 8 stages of the DevOps lifecycle in correct order
- ❖ Classify each stage as Dev-heavy or Ops-heavy
- ❖ Explain how monitoring feeds back into planning to close the loop
- ❖ Map a real product feature through all 8 stages

■ **ANALOGY**

DevOps lifecycle = a wheel, not a waterfall → code never "finishes" — it continuously improves

◆ DID YOU KNOW? ◆

8	Left 4	Right 4
stages in the infinite loop	Dev-heavy: Plan→Code→Build→Test	Ops-heavy: Release→Deploy→Operate→Monitor

The 8 Stages — Explained as Bullets

- **Plan:** Define what to build — backlog management
  - → → tools: Jira, Linear, GitHub Issues, Confluence, Notion
  - → → activities: user story writing, sprint planning, backlog grooming
  - → → output: a prioritised list of features and bug fixes to build
- **Code:** Write the software — version controlled
  - → → tools: VS Code, Git, GitHub, GitLab, Bitbucket
  - → → activities: feature development, code reviews, pull requests
  - → → output: reviewed, approved code merged to the main branch
- **Build:** Package code into a deployable artefact
  - → → tools: Maven, Gradle, npm, Docker, Bazel, Make
  - → → activities: compile source code, run static analysis, create Docker image
  - → → output: a versioned, immutable deployable artefact (e.g., Docker image, JAR file)
- **Test:** Verify correctness automatically — no manual QA
  - → → tools: JUnit, Pytest, Selenium, SonarQube, Trivy, k6
  - → → activities: unit tests, integration tests, security scans, performance tests
  - → → output: test results report — pass = proceed, fail = notify developer immediately
- **Release:** Approve and tag the production-ready version
  - → → tools: GitHub Releases, Jira, ServiceNow, Helm charts
  - → → activities: semantic versioning (v1.4.2), release notes, change management

→ → output: a tagged, documented release approved for production deployment

- **Deploy:** Put it on live servers — automated where possible
  → → tools: Jenkins, GitHub Actions, ArgoCD, AWS CodeDeploy, Spinnaker
  → → activities: blue/green deployment, canary release, rolling update
  → → output: new version running in production or staged for traffic
- **Operate:** Keep it running — scaling, reliability, config management
  → → tools: Kubernetes, Docker, Ansible, Terraform, AWS ECS
  → → activities: auto-scaling, failover, patching, capacity planning
  → → output: stable, available, well-performing service meeting SLA targets
- **Monitor:** Watch everything — metrics, logs, traces, alerts
  → → tools: Prometheus, Grafana, Datadog, CloudWatch, Jaeger, PagerDuty
  → → activities: dashboards, alerting, SLO tracking, error rate analysis
  → → output: insights that feed back into the next Plan stage — closing the loop

Stage	Side	Key Tools	Output
Plan	Dev	Jira, Linear, GitHub Issues	Prioritised backlog
Code	Dev	Git, GitHub, VS Code	Reviewed, merged code
Build	Dev	Docker, Maven, Gradle	Versioned artefact
Test	Dev	JUnit, Pytest, SonarQube	Test results pass/fail
Release	Both	GitHub Releases, Helm	Tagged release version
Deploy	Ops	GitHub Actions, ArgoCD	Live deployment
Operate	Ops	Kubernetes, Ansible	Stable running service
Monitor	Ops	Prometheus, Grafana	Insights → next Plan

■ TRY IT YOURSELF

1. Pick any mobile app you use — trace its last update through all 8 DevOps lifecycle stages
2. Create a GitHub repository → open an Issue (Plan) → commit a README file (Code) — you just did 2 stages!
3. Look up "GitHub Actions quickstart" and set up your first automated build (Stage 3: Build) using a free repo

✓ QUICK SUMMARY

- **8 stages:** Plan → Code → Build → Test → Release → Deploy → Operate → Monitor
- **Dev-heavy (left):** Plan, Code, Build, Test — focused on building features
- **Ops-heavy (right):** Release, Deploy, Operate, Monitor — focused on running services
- **The loop never ends:** Monitor feeds insights back into Plan — continuous improvement
- **CI/CD sits at the heart:** automating the Build → Test → Deploy handoffs
- **Key mindset:** software is never "done" — every deployment begins the next iteration

■ MINI QUIZ — Test Your Understanding

Q1. Which stage of the DevOps lifecycle feeds back into the very first stage?

a) Deploy — you deploy back to Plan

b) Test — failures go back to Code

c) Monitor — production insights drive the next Plan

d) Release — version numbers restart

✓ Answer: c — Monitor closes the loop — real user behaviour and error rates inform what to build next in Plan

Q2. A developer wants to check if new code breaks existing functionality automatically. Which stage handles this?

a) Plan

b) Code

c) Test

d) Deploy

✓ Answer: c — The Test stage runs automated unit, integration, and regression tests on every build

Q3. Which tools are used in the Operate stage?

a) Jira and Confluence

b) Kubernetes, Docker, and Ansible

c) Prometheus and Grafana

d) GitHub and VS Code

✓ Answer: b — Kubernetes and Ansible manage running infrastructure — Prometheus/Grafana are for Monitor stage

CHAPTER 06 ■ Intermediate ■ 10 min read 6/14

CI/CD — Continuous Integration & Delivery

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

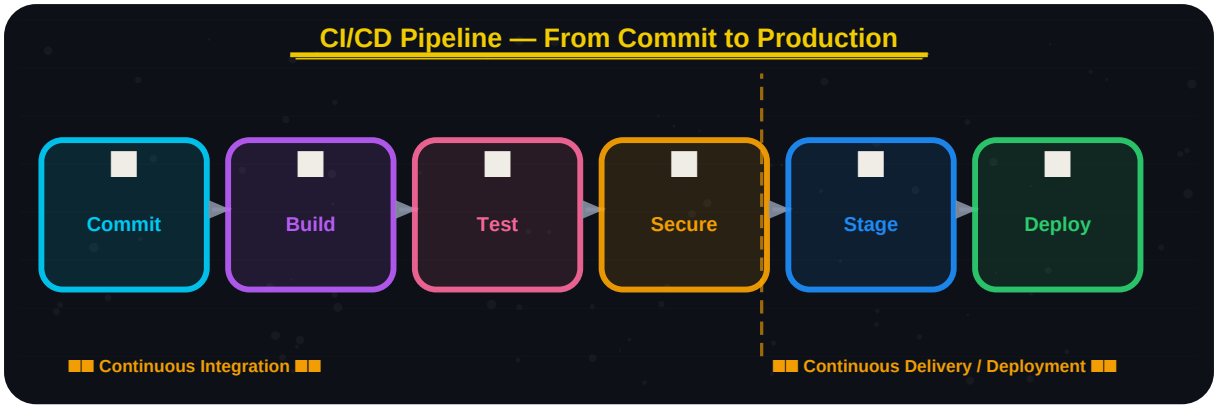
- ❖ Distinguish between CI, Continuous Delivery, and Continuous Deployment
- ❖ Explain what a pipeline is and what stages it contains
- ❖ Describe the "Fail Fast" principle and why it saves money
- ❖ Give an example of CI/CD enabling a real company to ship faster and safer

■ ANALOGY

CI/CD = factory assembly line → raw code in → tested, packaged software out → automatically

◆ DID YOU KNOW? ◆

5 min	Zero	100x
Facebook CI runs 1000+ tests in	manual server touches in mature CI/CD	cheaper to fix bugs in CI vs production



CI/CD Pipeline — the automated path from every code commit to production deployment

CI/CD — Three Concepts Clearly Defined

- **Continuous Integration (CI):** merge code early and often
  - → → developers push code to a shared branch multiple times per day
  - → → every push automatically triggers: build + automated tests
  - → → goal: catch integration conflicts within minutes, not after weeks
  - → → eliminates "Integration Hell" — small merges are trivial to resolve
- **Continuous Delivery (CD):** always have a deployable build
  - → → every passing CI build is automatically packaged as a release candidate
  - → → deployment to production requires: one human approval button click
  - → → business can choose WHEN to release — the system is always ready
  - → → good for: regulated industries where humans must sign off on releases

- **Continuous Deployment (CD):** zero human intervention to production
  → → every passing build automatically deploys to production — no button click
  → → requires: very high test coverage, automated rollback, strong monitoring
  → → companies using this: Netflix, Amazon, GitHub, Etsy
  → → most mature level — requires months of CI/CD maturity to achieve safely

Key CI/CD Concepts

- **Pipeline:** the ordered sequence of automated stages code passes through → Commit → Build → Test → Deploy
- **Build Automation:** compiling, packaging, and containerising code without any manual steps
- **Automated Testing:** unit tests, integration tests, end-to-end tests, and security scans — no human QA required
- **Fail Fast:** detect problems at the earliest possible stage → bugs in CI = cheap fix; bugs in production = expensive crisis
- **Artefact:** the output of a successful build — a Docker image, a JAR file, a compiled binary
- **Rollback:** automatic reversal to previous version if error rates spike after deployment

■ REAL-WORLD EXAMPLE

Meta (Facebook)

Situation:

• Needed to push hundreds of code changes daily to billions of users without downtime

What they did:

• Built an internal CI system that runs 10,000+ automated tests in parallel for every commit
• Implemented "Facebook Landcastle" — a pre-merge testing system that catches 85% of production bugs
• Used feature flags to deploy code to 1% of users first, then gradually roll out globally

Result:

• Push thousands of changes per day with less than 0.001% user-visible error rate
• 5-minute CI pipeline catches issues that previously reached production
• Engineers can confidently push code at any time, including Fridays

■ TRY IT YOURSELF

1. Create a GitHub repo → add a .github/workflows/ci.yml file → trigger your first GitHub Actions CI pipeline
2. Add a simple Python test (test_math.py with one assertion) → watch the CI pipeline run and pass automatically
3. Break the test intentionally → commit → observe how the pipeline fails and blocks the "deployment"

✓ **QUICK SUMMARY**

- **CI:** merge small code changes frequently → auto build and test on every push
- **Continuous Delivery:** every passing build is deployment-ready → human approves final step
- **Continuous Deployment:** fully automated → every passing build goes live automatically
- **Pipeline:** Commit → Build → Test → Security Scan → Stage → Deploy → Monitor
- **Fail Fast:** find bugs in CI (cheap) not in production (expensive and customer-facing)
- **Rollback:** automatic reversal is essential before enabling Continuous Deployment

■ **MINI QUIZ — Test Your Understanding**

Q1. What is the key difference between Continuous Delivery and Continuous Deployment?

a) Continuous Delivery skips testing

b) Continuous Deployment requires human approval; Delivery does not

c) Continuous Delivery requires human approval; Deployment deploys automatically

d) They are the same thing

✓ Answer: c — Continuous Delivery = human approves the final production push. Continuous Deployment = fully automated, no human needed

Q2. A bug is found in production. When was the cheapest time to have caught this bug?

a) After customer reports it

b) During manual QA

c) During automated CI testing after the commit

d) During the staging environment test

✓ Answer: c — The Fail Fast principle: bugs caught at CI stage cost ~$100 to fix; same bug in production can cost $100,000+

Q3. Which tool is most commonly used for CI/CD pipelines in 2026?

a) Microsoft Word

b) GitHub Actions or Jenkins

c) Slack or Teams

d) Google Sheets

✓ Answer: b — GitHub Actions and Jenkins are the industry-standard CI/CD tools — GitHub Actions has surged in adoption since 2020

CHAPTER 07 ■ Intermediate ■ 9 min read 7/14

Automation & Infrastructure as Code (IaC)

■ **LEARNING OBJECTIVES**

By the end of this chapter you will be able to:

- ❖ Explain Infrastructure as Code in one sentence without jargon
- ❖ Describe the difference between Terraform (provision) and Ansible (configure)
- ❖ Give 3 advantages of IaC over manual server setup
- ❖ Explain what "idempotent" means in the context of IaC

■ **ANALOGY**

IaC = LEGO instruction manual → anyone can rebuild identical infrastructure from the same code file

◆ DID YOU KNOW? ◆

4 min	0	100%
Terraform provisions 50 servers in	manual console clicks needed with IaC	reproducible — same code, same result

What is Infrastructure as Code?

- **Infrastructure as Code (IaC):** managing servers, networks, and cloud resources through version-controlled text files
  - → → instead of: clicking through AWS console for hours
  - → → you write: a Terraform .tf file describing what should exist
  - → → run: terraform apply → infrastructure created in minutes
- **Idempotent:** running the same code twice produces the same result — no duplicates
  - → → run terraform apply 10 times → get same infrastructure as running it once
  - → → critical for reliability — scripts should be safe to re-run

Key Advantages of IaC

- **Eliminates snowflake servers:** every server built from code is identical → no "it works on server-A but not server-B"
- **Version controlled:** infrastructure changes tracked in Git → "git blame" works on your servers
- **Speed:** 50 servers provisioned in 4 minutes vs 3 days of manual clicking
- **Disaster recovery:** entire infrastructure rebuilt from code in minutes — not weeks of manual rebuilding
- **Environment parity:** Dev, Staging, and Production are identical → "works in staging" = works in production

- **Cost control:** spin up for testing, destroy when done → no idle billing overnight

Tool	Category	What It Does	Key Difference
Terraform	Provisioning	Creates/destroys cloud resources: EC2, VPCs, databases	Declarative: you define end state
Ansible	Configuration	Installs software, configures settings on existing servers	Agentless: uses SSH only
Pulumi	Provisioning	Same as Terraform but in Python/TypeScript/Go — no DSL	Code-first approach
CloudFormation	Provisioning	Terraform for AWS only — JSON or YAML templates	AWS-native, no extra install
Chef / Puppet	Configuration	Older agent-based config management tools	Agent must be on every server

■ REAL-WORLD EXAMPLE

Shopify

Situation:

- Black Friday 2023: needed to provision hundreds of additional servers in hours, not days

What they did:

- Ran a single Terraform command: terraform apply -var="instance_count=400"
- Ansible automatically configured each server with the latest application version
- After Black Friday: terraform destroy removed all 400 servers in one command

Result:

- 400 servers provisioned and configured in under 12 minutes
- Zero manual console clicks — entire process was a 2-line script

■ TRY IT YOURSELF

1. Install Terraform: brew install terraform (Mac) or visit terraform.io/downloads for Windows
2. Write a 5-line Terraform file to create an AWS S3 bucket → run terraform plan to preview it (no credit card needed)
3. Compare: time yourself clicking through the AWS console to create an S3 bucket vs running terraform apply — which is faster and more repeatable?

✓ QUICK SUMMARY

- **IaC:** describe infrastructure in code files → version controlled, repeatable, automated
- **Idempotent:** run the same IaC code 10 times → identical result every time, no duplicates
- **Terraform:** provisions what should exist — creates, modifies, destroys cloud resources
- **Ansible:** configures what is running — installs software, sets config, manages state
- **Key benefit:** Dev, Staging, Production are identical → eliminates environment-specific bugs
- **Disaster recovery:** entire infrastructure rebuilt from a git clone + terraform apply

■ MINI QUIZ — Test Your Understanding

Q1. What does "idempotent" mean in the context of IaC?

a) The code runs very fast

b) Running the same code multiple times always produces the same result

c) The infrastructure costs nothing

d) The code works on any cloud provider

✓ Answer: b — Idempotent means: apply 1 time or 100 times → same result. This makes IaC scripts safe to re-run without creating duplicates

Q2. What is the key difference between Terraform and Ansible?

a) Terraform is older than Ansible

b) Terraform provisions infrastructure; Ansible configures software on existing infrastructure

c) Ansible provisions infrastructure; Terraform configures software

d) They do exactly the same thing

✓ Answer: b — Terraform = what infrastructure should exist (VMs, networks, databases). Ansible = what software and config should be on those machines

Q3. A company needs identical Dev, Staging, and Production environments. Which approach achieves this best?

a) Manually configure each environment the same way

b) Take screenshots of the console settings and copy them

c) Use the same IaC code (Terraform + Ansible) for all three environments

d) Only worry about Production — Dev can be different

✓ Answer: c — IaC with the same codebase across environments guarantees identical, reproducible infrastructure — the foundation of reliable deployments

CHAPTER 08 ■ Beginner ■ 8 min read 8/14

Introduction to Cloud Computing

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ❖ Explain cloud computing in one sentence a non-technical person would understand
- ❖ List the 5 key characteristics of cloud computing
- ❖ Compare AWS, Azure, and GCP at a high level
- ❖ Explain the difference between CapEx and OpEx in the context of cloud

■ ANALOGY

Cloud = renting a fully-managed apartment vs buying a house → no maintenance, pay monthly, move instantly

◆ DID YOU KNOW? ◆

260M+	$480B	3
Netflix users served on AWS	global cloud market by 2028 (Gartner)	major providers: AWS, Azure, GCP

What is Cloud Computing?

- **Cloud computing:** delivery of computing resources over the internet — on demand, at any scale
  - → → instead of: buying and maintaining your own servers
  - → → you rent: compute, storage, databases, AI, networking from providers
  - → → pay: only for what you use — like electricity, not a mortgage

5 Key Characteristics (NIST Definition)

- **On-demand self-service:** get a server in 90 seconds — no procurement approval, no waiting weeks for hardware
- **Broad network access:** access resources from anywhere via internet — laptop, phone, or automated scripts
- **Resource pooling:** providers share infrastructure across thousands of customers — isolation via virtualisation
- **Rapid elasticity:** scale up in seconds during peak demand → scale down automatically at 3am to save cost
- **Measured service:** pay-as-you-go metering — billed per second, per GB, or per API call

CapEx vs OpEx — Why Cloud Changes Finance

- **CapEx (Capital Expenditure):** buying servers upfront → large cost today, depreciate over 5 years → traditional IT

- **OpEx (Operating Expenditure):** renting cloud resources monthly → pay-as-you-go → cloud model
- **Cloud advantage:** no upfront capital → start with $0, scale as revenue grows → ideal for startups and enterprises
- **Warning:** OpEx can become expensive if resources are not managed → active FinOps practice required

Provider	Market Share	Key Strengths	Best Known For
AWS (Amazon)	~32%	Broadest service catalogue, highest job demand, most mature	EC2, S3, Lambda, EKS, RDS
Azure (Microsoft)	~22%	Enterprise/Windows, Microsoft 365 integration, hybrid cloud	VMs, AKS, Azure Functions, Entra ID
GCP (Google)	~11%	AI/ML leadership, data analytics, Kubernetes (they created it)	GKE, BigQuery, Vertex AI, Cloud Run

■ **REAL-WORLD EXAMPLE** — **Netflix**

Situation:

- Needed to stream to 260M+ users globally without owning any data centres

What they did:

- Migrated entirely to AWS — zero owned data centres
- Uses AWS Auto Scaling to match server count to real-time viewership demand
- Deploys to multiple AWS regions simultaneously for global low-latency streaming

Result:

- Handles 150TB+ of data served daily from AWS infrastructure
- Scales from 10M viewers at 6am to 100M+ at 9pm automatically — no human intervention

■ **COMMON MISCONCEPTIONS**

■ **Myth:**

"The cloud" is somewhere in the sky

■ **Reality:**

Cloud = physical servers in ground-level warehouses (data centres) in specific geographic regions — completely physical

■ **Myth:**

"My data is automatically safe in the cloud"

■ **Reality:**

Cloud providers secure the physical infrastructure (Shared Responsibility Model) — YOU are responsible for securing your data, access controls, and configurations

■ TRY IT YOURSELF

1. Create a free AWS account at aws.amazon.com/free — no credit card required for most free tier services
2. Launch a t2.micro EC2 instance (free tier) → note how fast it appears vs ordering physical hardware
3. Calculate: AWS t3.micro costs ~$0.0104/hour → what would 1 year of 24/7 running cost? Compare to buying a server

✓ QUICK SUMMARY

- **Cloud:** computing resources rented over the internet — no owned hardware, pay per use
- **5 characteristics:** on-demand, broad access, resource pooling, rapid elasticity, measured service
- **CapEx → OpEx:** cloud shifts from buying hardware upfront to renting monthly
- **AWS:** largest provider — most jobs, broadest catalogue — ideal for DevOps learning
- **Shared Responsibility:** provider secures infrastructure — YOU secure your data and configs
- **Free tier:** AWS, Azure, and GCP all offer free tiers — start building today for free

■ MINI QUIZ — Test Your Understanding

Q1. A startup wants to launch a web app with zero upfront investment and scale as users grow. Which model fits best?

a) Buy servers upfront (CapEx)

b) Use cloud computing (OpEx)

c) Rent a data centre

d) Use only localhost

✓ Answer: b — Cloud OpEx model lets startups pay only for what they use and scale resources as revenue grows — no upfront CapEx needed

Q2. Netflix uses AWS and needs to handle 10x more viewers on Christmas Day vs a Tuesday morning. What cloud feature enables this?

a) Cloud is always the same size

b) They buy extra servers before Christmas

c) Rapid elasticity — auto-scale up instantly and down automatically

d) They cache everything locally on user devices

✓ Answer: c — Rapid elasticity is a core cloud characteristic — resources scale automatically based on real-time demand

Q3. Who is responsible for securing the data stored in an AWS S3 bucket?

a) AWS — they own the infrastructure

b) The customer — they own their data and access controls

c) AWS secures the bucket; customer secures the internet

d) No one — cloud data is inherently public

✓ Answer: b — The Shared Responsibility Model: AWS secures the infrastructure; the customer secures their data, IAM permissions, and bucket policies

CHAPTER 09 ■ Beginner ■ 7 min read 9/14

Why Cloud is Needed

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ❖ Describe the traditional server capacity planning problem and its consequences
- ❖ List 5 key business benefits of cloud computing with real examples
- ❖ Explain why cloud requires active cost management to avoid bill shock
- ❖ Describe how Zoom used cloud to scale 30x in 3 weeks

■ ANALOGY

Cloud capacity = accordion → expands instantly under demand, contracts automatically when quiet

◆ DID YOU KNOW? ◆

30x	90 sec	99.99%
Zoom scaled users in 3 weeks (COVID)	to provision a server vs 6 weeks on-premise	uptime SLA from top cloud providers

The Traditional Capacity Problem

- **The impossible choice:** buy too few servers OR buy too many
  - → → too few: site crashes under traffic spikes → lost revenue, angry customers
  - → → too many: idle servers cost money 24/7 → massive wasted CapEx
  - → → cloud solution: pay for exactly what you use, scale in real-time

5 Key Business Benefits

- **Scalability:** grow or shrink resources based on actual demand
  - → → vertical scaling: make a server bigger (more CPU/RAM)
  - → → horizontal scaling: add more servers automatically
  - → → example: e-commerce adds 100 servers at 8am on Black Friday, removes them at 10pm
- **Speed to market:** developers get resources in minutes, not weeks
  - → → traditional: submit ticket → procurement → delivery → racking → OS install → 6 weeks
  - → → cloud: aws ec2 run-instances → server ready in 90 seconds
  - → → faster infrastructure = faster features = faster competitive advantage
- **Reliability:** built-in redundancy at every level
  - → → multiple Availability Zones (AZs) — physically separate data centres
  - → → if one AZ fails → traffic automatically reroutes to another
  - → → providers offer 99.99% SLA = max 52 minutes downtime per year
- **Cost efficiency:** OpEx model eliminates waste

→ → pay per second/minute/hour — no idle billing for unused capacity

→ → reserved instances: commit 1-3 years → up to 72% discount vs on-demand

→ → spot instances: bid for unused capacity → up to 90% discount for non-critical work

- **Global reach:** deploy worldwide in one click

→ → AWS has 30+ regions across 6 continents

→ → deploy to Tokyo, London, and São Paulo simultaneously with one Terraform command

→ → users served from the nearest region → lower latency → better experience

■ **REAL-WORLD EXAMPLE** Zoom

Situation:

• COVID-19 March 2020: daily users exploded from 10M to 300M in 3 weeks — a 30x increase

What they did:

• Used AWS and Oracle Cloud elastic infrastructure to add capacity dynamically

• Scaled video-processing servers by hundreds per day using cloud auto-scaling

• Deployed to additional cloud regions to handle global demand simultaneously

Result:

• Served 300M daily participants without a service collapse

• 30x scale achieved in 3 weeks — impossible with physical data centres

■ **COMMON MISCONCEPTIONS**

■ **Myth:**

"Cloud is always cheaper than on-premise"

■ **Reality:**

Cloud costs can exceed on-premise if resources are left running unnecessarily. Active FinOps (cloud cost management) is essential — set budget alerts, rightsize instances, use reserved capacity

■ **Myth:**

"Cloud availability zones are just marketing — there is no real redundancy"

■ **Reality:**

AWS Availability Zones are physically separate data centres with independent power, cooling, and networking — a fire in one AZ does not affect another

■ **TRY IT YOURSELF**

1. Go to calculator.aws → estimate the monthly cost of 2 t3.medium EC2 instances running 24/7 in us-east-1
2. Search "AWS Auto Scaling documentation" → read how a scaling policy works and draw a simple diagram of the flow
3. Find the AWS global infrastructure map at aws.amazon.com/about-aws/global-infrastructure → count how many regions exist today

✓ **QUICK SUMMARY**

- **Traditional problem:** buy too few (crashes) or too many (waste) — cloud solves this
- **Scalability:** grow or shrink in seconds based on real demand — no capacity guessing
- **Speed:** server in 90 seconds vs 6 weeks procurement → faster products to market
- **Reliability:** multiple AZs, auto-failover, 99.99% SLA — higher than most on-premise
- **Global reach:** 30+ AWS regions → serve users from the nearest data centre worldwide
- **Cost warning:** cloud is not automatically cheaper — active FinOps management required

■ **MINI QUIZ — Test Your Understanding**

Q1. Zoom grew from 10M to 300M daily users in 3 weeks. What cloud characteristic made this possible?

a) On-demand self-service and rapid elasticity

b) They bought 290M extra servers in advance

c) They reduced video quality to fit more users

d) Cloud providers gave them unlimited free resources

✓ Answer: a — On-demand self-service + rapid elasticity = scale from 10M to 300M users in 3 weeks without any manual provisioning

Q2. A company leaves 100 cloud servers running at night when nobody uses them. What is the problem?

a) The servers will overheat

b) They are paying for idle cloud resources — wasting money

c) The cloud provider will charge extra for night-time use

d) This is normal and expected behaviour

✓ Answer: b — Cloud charges for all running resources — idle servers cost as much as active ones. FinOps: turn off or scale down unused resources

Q3. What is an AWS Availability Zone (AZ)?

a) A marketing zone for selling AWS services

b) A physically separate data centre with independent power and cooling

c) A virtual boundary with no physical significance

d) An AWS feature for billing purposes only

✓ Answer: b — AZs are physically distinct data centres within a region — separate power, cooling, networking. Deploying across AZs provides true high availability

CHAPTER 10 ■ Beginner ■ 9 min read 10/14

Cloud Service Models — IaaS, PaaS, SaaS

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ❖ Explain IaaS, PaaS, and SaaS using the pizza analogy in your own words
- ❖ Map a real service (Heroku, Gmail, AWS EC2) to its correct model
- ❖ Describe what YOU manage vs what the PROVIDER manages in each model
- ❖ Explain why AWS is not purely IaaS — it spans all three models

■ ANALOGY

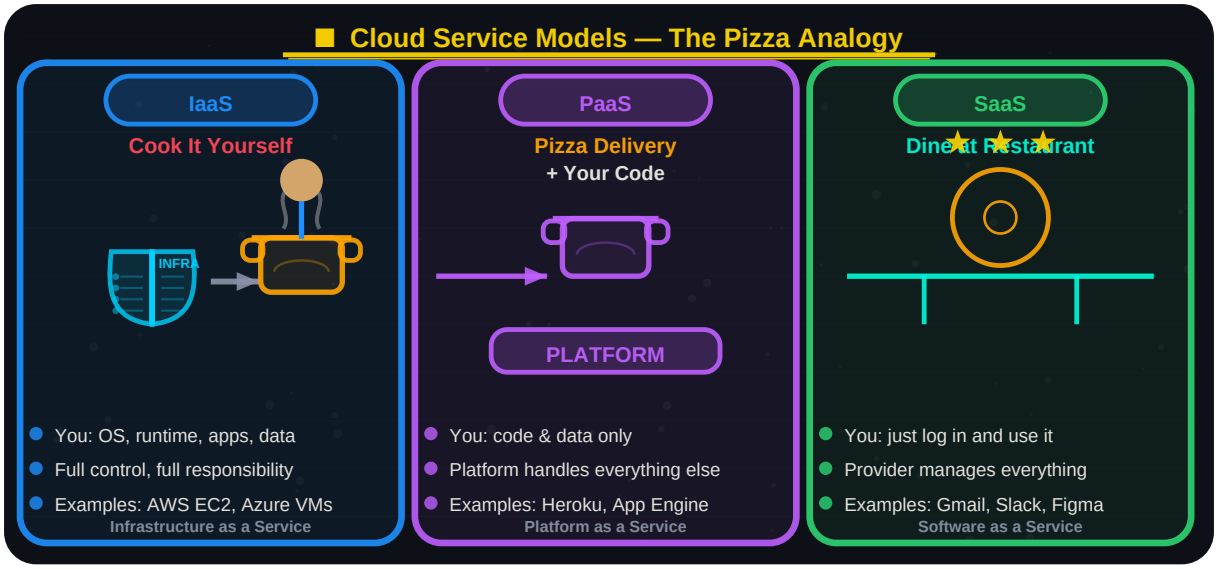
IaaS = cook at home · PaaS = pizza delivery · SaaS = restaurant dining — you manage less each time

◆ DID YOU KNOW? ◆

IaaS — Full control full responsibility

PaaS — Build without managing the infrastructure

SaaS — Just log in and use it



IaaS = cook yourself · PaaS = pizza delivery · SaaS = dine at restaurant — responsibility decreases left to right

IaaS, PaaS, SaaS — Clear Bullet Definitions

- **IaaS (Infrastructure as a Service):** rent raw hardware — you manage everything above
  - → → you manage: OS, runtime, middleware, application, data
  - → → provider manages: physical servers, networking, storage, power
  - → → analogy: rent a fully equipped kitchen — you cook everything from scratch
  - → → examples: AWS EC2, Azure Virtual Machines, Google Compute Engine
  - → → best for: teams needing full control (custom OS, specific GPU, legacy apps)

- **PaaS (Platform as a Service):** deploy code without managing infrastructure
  - → → you manage: application code and data only
  - → → provider manages: OS, runtime, middleware, scaling, patching
  - → → analogy: pizza delivery — platform brings ready dough, you add your toppings (code)
  - → → examples: Heroku, Google App Engine, AWS Elastic Beanstalk, Railway
  - → → best for: developers who want to ship code fast without DevOps overhead
- **FaaS (Function as a Service / Serverless):** run individual functions without any servers
  - → → you manage: function code and business logic only
  - → → provider manages: everything — servers, scaling, availability, runtime
  - → → analogy: ordering one specific dish from a menu — kitchen is invisible
  - → → examples: AWS Lambda, Azure Functions, Google Cloud Run
  - → → best for: event-driven workloads (image processing, API calls, scheduled jobs)
- **SaaS (Software as a Service):** use finished software over the internet
  - → → you manage: your data and settings only
  - → → provider manages: everything — infrastructure, software, updates, security
  - → → analogy: dining at a restaurant — you just order, eat, and pay
  - → → examples: Gmail, Slack, Salesforce, GitHub, Figma, Jira
  - → → best for: end users who need software without any technical management

Model	Examples	You Manage	Provider Manages	Best For
IaaS	AWS EC2, Azure VMs	OS, Runtime, Apps, Data	Hardware, Network, Storage	Custom workloads, full control
PaaS	Heroku, App Engine	App code and data only	OS, Runtime, Middleware	Developer speed, no DevOps overhead
FaaS	Lambda, Cloud Run	Function code only	Everything else	Event-driven, pay-per-execution
SaaS	Gmail, Slack, GitHub	Your data + settings	The entire application	End users, zero infrastructure concern

■ **COMMON MISCONCEPTIONS**

■ **Myth:**

"AWS is just IaaS" — AWS is one type of cloud service

■ **Reality:**

AWS spans ALL models: EC2 = IaaS, Elastic Beanstalk = PaaS, Lambda = FaaS, WorkMail = SaaS. The service you choose determines the model

■ **Myth:**

"More control is always better — use IaaS for everything"

■ **Reality:**

More control = more responsibility. A startup using IaaS for a simple blog spends 80% of time on infrastructure instead of building product. PaaS/SaaS let you focus on value

■ TRY IT YOURSELF

1. Deploy a "Hello World" Python Flask app to Heroku (PaaS) — compare the effort vs setting up your own EC2 server (IaaS)
2. Create an AWS Lambda function (FaaS) that returns "Hello World" — observe: zero servers to manage, pay only when triggered
3. List 5 SaaS tools you use daily → estimate what building each from scratch would cost in developer time

✓ QUICK SUMMARY

- **IaaS:** rent raw servers — you manage OS and everything above (AWS EC2, Azure VMs)
- **PaaS:** deploy code without managing infrastructure (Heroku, App Engine)
- **FaaS/Serverless:** run individual functions — zero servers (AWS Lambda, Cloud Run)
- **SaaS:** use finished software over the internet — just log in (Gmail, Slack, Jira)
- **Trade-off:** more control (IaaS) → more responsibility; more convenience (SaaS) → less control
- **AWS spans all models:** EC2 (IaaS), Beanstalk (PaaS), Lambda (FaaS), WorkMail (SaaS)

■ MINI QUIZ — Test Your Understanding

Q1. A developer uploads Python code to Heroku with "git push heroku main" and it runs — no server setup. What model is this?

a) IaaS — they are using virtual machines
b) SaaS — they are using software
c) PaaS — the platform handles infrastructure
d) FaaS — it is serverless

✓ Answer: c — Heroku is PaaS — you push code, the platform handles OS, runtime, scaling, and deployment automatically

Q2. An AWS Lambda function costs $0.0000002 per request and charges nothing when idle. What model is this?

a) IaaS
b) PaaS
c) SaaS
d) FaaS/Serverless

✓ Answer: d — FaaS/Serverless: billed per invocation, zero cost when idle, zero servers to manage — AWS Lambda is the most popular example

Q3. Which statement is TRUE about SaaS?

a) You manage the operating system
b) You manage the database server
c) You manage only your data and account settings
d) You must install the software on your own servers

✓ Answer: c — SaaS = you manage nothing except your own data and settings. The provider handles everything: infrastructure, software, updates, security

CHAPTER 11 ■ Beginner ■ 7 min read 11/14

Cloud Deployment Models

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ❖ Explain Public, Private, Hybrid, and Multi-Cloud in plain English
- ❖ Give a real-world scenario where each deployment model is the right choice
- ❖ Describe why most large enterprises use Hybrid or Multi-Cloud today
- ❖ Explain the trade-offs between Public Cloud cost and Private Cloud security

■ ANALOGY

Public Cloud = public bus · Private Cloud = personal car · Hybrid = car + bus together · Multi = using Uber + bus + train

◆ DID YOU KNOW? ◆

72%	Private	Hybrid
of enterprises use multi-cloud strategy	= highest security highest cost	= most common enterprise model

4 Cloud Deployment Models

- **Public Cloud:** shared infrastructure managed by providers
  - → → resources shared across thousands of customers, isolated by virtualisation
  - → → cost: cheapest option — pay only for what you use
  - → → managed by: AWS, Azure, GCP — no hardware ownership required
  - → → best for: startups, SaaS products, development/test environments, global scaling
  - → → examples: Netflix on AWS, Spotify on GCP, LinkedIn on Azure
- **Private Cloud:** dedicated infrastructure for one organisation
  - → → servers dedicated exclusively to your organisation — no resource sharing
  - → → deployed in: your own data centre OR a colocation facility
  - → → cost: most expensive — you buy, maintain, and operate all hardware
  - → → best for: banking, healthcare, government with strict data sovereignty laws
  - → → examples: VMware vSphere, OpenStack, on-premise Kubernetes clusters
- **Hybrid Cloud:** public + private connected securely
  - → → sensitive data: kept in private cloud (compliance requirement)
  - → → scalable workloads: run on public cloud (cost and agility benefit)
  - → → connected via: VPN or dedicated link (AWS Direct Connect, Azure ExpressRoute)
  - → → best for: regulated enterprises that need both security and scalability
  - → → examples: HSBC (private for customer data, public for marketing)
- **Multi-Cloud:** multiple cloud providers used simultaneously
  - → → no vendor lock-in: not dependent on one provider's pricing or outages

→ → best-of-breed: use AWS for compute, GCP for AI/ML, Azure for Microsoft integration

→ → risk mitigation: if AWS has an outage, workloads fail over to Azure automatically

→ → complexity: requires unified management tools (Terraform, Crossplane)

→ → examples: Airbnb (AWS + GCP), Spotify (GCP + AWS)

Model	Owned By	Cost	Security	Best For
Public Cloud	Provider (AWS/Azure/GCP)	Lowest	Good — shared but isolated	Startups, agile scaling, SaaS
Private Cloud	Your organisation	Highest	Highest — fully dedicated	Banking, healthcare, government
Hybrid Cloud	Both	Medium	High — sensitive data on-prem	Regulated enterprise with cloud burst
Multi-Cloud	Multiple providers	Varies	Depends on config	Avoid lock-in, best-of-breed services

■ **REAL-WORLD EXAMPLE** — **HSBC Bank**

Situation:

- Needed strict compliance for financial data while wanting cloud agility for customer-facing services

What they did:

- Kept customer financial records and transaction data in private on-premise infrastructure
- Ran public website, analytics, and marketing tools on AWS and Google Cloud
- Connected private and public environments via AWS Direct Connect (1Gbps dedicated link)

Result:

- Met FCA and HKMA regulatory requirements for data sovereignty
- Saved 30% on operational costs vs running everything on-premise
- Classic Hybrid Cloud — the standard architecture for regulated financial institutions

■ **TRY IT YOURSELF**

1. Research "AWS Outposts" — how does it bring AWS public cloud services to your own private data centre?
2. Find one real Hybrid Cloud case study from a bank or hospital on Google → document why they chose Hybrid
3. Draw a simple diagram of a Hybrid Cloud setup: on-prem server ↔ VPN ↔ AWS VPC — label the components

✓ **QUICK SUMMARY**

- **Public:** shared provider infrastructure — cheapest, most agile (AWS/Azure/GCP)
- **Private:** dedicated hardware — most secure, most expensive, self-managed
- **Hybrid:** public + private connected — regulated data on-prem + scalable workloads on cloud
- **Multi-Cloud:** multiple providers — avoids lock-in, uses best service per workload
- **72% of enterprises:** use multi-cloud strategy today — rarely just one provider
- **Key trade-off:** public = cost and agility vs private = security and compliance

■ **MINI QUIZ — Test Your Understanding**

Q1. A hospital keeps patient records on private servers but uses AWS for their appointment booking website. This is:

a) Public Cloud — they use AWS

b) Private Cloud — they own servers

c) Hybrid Cloud — private for sensitive data, public for scalable workloads

d) Multi-Cloud — they use multiple services

✓ Answer: c — Hybrid Cloud = sensitive/regulated data on private infrastructure + scalable/public workloads on cloud — classic for healthcare and finance

Q2. A company uses AWS for compute, GCP for BigQuery analytics, and Azure for Active Directory. This is:

a) Public Cloud

b) Private Cloud

c) Hybrid Cloud

d) Multi-Cloud

✓ Answer: d — Multi-Cloud = using multiple cloud providers simultaneously — AWS + GCP + Azure at the same time

Q3. Which deployment model offers the LOWEST cost?

a) Private Cloud

b) Hybrid Cloud

c) Public Cloud

d) Multi-Cloud is always cheapest

✓ Answer: c — Public Cloud: shared infrastructure costs spread across thousands of customers → lowest per-unit price. Private Cloud is most expensive (you own all hardware)

CHAPTER 12 ■ Intermediate ■ 8 min read 12/14

DevOps + Cloud — Stronger Together

■ **LEARNING OBJECTIVES**

By the end of this chapter you will be able to:

- ❖ Explain the relationship between DevOps (method) and Cloud (toolbox)
- ❖ Describe how cloud amplifies each of the 5 DevOps practices
- ❖ Define Cloud-Native, Serverless, and GitOps in bullet format
- ❖ Give a real example of DevOps + Cloud delivering business value at scale

■ **ANALOGY**

DevOps = HOW you work · Cloud = WHERE you work → together = maximum speed, scale, and reliability

◆ DID YOU KNOW? ◆

DevOps	Cloud	Together
is the METHOD how you work	is the TOOLBOX where you work	fastest engineering in history

How DevOps and Cloud Amplify Each Other

- **IaC + Cloud APIs:** infrastructure becomes fully programmable
  - → → Terraform calls AWS/Azure/GCP APIs to provision 1,000 resources in seconds
  - → → no IaC without cloud APIs — cloud makes infrastructure as manageable as code
- **CI/CD + Cloud Compute:** elastic build infrastructure
  - → → GitHub Actions spins up fresh cloud VMs for every build → no dedicated build servers
  - → → parallel test runs across 100 cloud VMs simultaneously → 5-minute pipelines
- **Containers + Managed Kubernetes:** run anywhere, scale instantly
  - → → EKS (AWS), GKE (Google), AKS (Azure) manage Kubernetes clusters for you
  - → → DevOps teams deploy containers without managing cluster infrastructure
- **Monitoring + Cloud Native Tools:** observability at scale
  - → → CloudWatch (AWS), Azure Monitor, Google Cloud Monitoring integrate natively
  - → → Prometheus + Grafana runs on cloud-managed services — zero server management
- **GitOps + Cloud:** Git as single source of truth for everything
  - → → ArgoCD watches Git → automatically applies changes to cloud Kubernetes clusters
  - → → every infrastructure and app change is a Git commit — full audit trail

Key Cloud-Native Concepts

- **Cloud-Native:** applications designed from day one for cloud → containers + microservices + managed services

- **Serverless:** run code without any server management → AWS Lambda charges per invocation, zero idle cost
- **GitOps:** Git = single source of truth for both app code AND infrastructure → ArgoCD, Flux
- **FinOps:** cloud financial management → rightsizing, budget alerts, reserved instances, cost attribution
- **Lift-and-Shift:** moving existing apps to cloud unchanged — NOT cloud-native → misses most cloud benefits

■ REAL-WORLD EXAMPLE

Netflix

Situation:

• Needed to serve 260M+ subscribers globally with zero scheduled maintenance windows

What they did:

• Moved to cloud-native microservices on AWS — 700+ independent services

• Practices Chaos Engineering: intentionally kills production servers to verify resilience

• Uses DevOps with full CD — code deploys continuously without any human approval

Result:

• Zero planned maintenance windows — updates roll out invisibly to subscribers

• New video codecs deploy to 100% of users within hours — tested automatically

■ COMMON MISCONCEPTIONS

■ **Myth:**

"DevOps and Cloud are the same thing"

■ **Reality:**

DevOps is HOW you work (culture, methodology, automation). Cloud is WHERE you work (computing environment). You can do DevOps on bare-metal servers. You can use cloud without DevOps

■ **Myth:**

"Cloud-Native just means running in the cloud"

■ **Reality:**

Cloud-Native means purpose-built for cloud from the start: containers, microservices, managed services, auto-scaling. Simply moving an old app to a VM is "lift-and-shift" — not cloud-native

■ TRY IT YOURSELF

1. Deploy a Docker container to AWS ECS Fargate (serverless containers) → observe: zero EC2 servers to manage
2. Set up a GitHub Actions workflow that runs terraform plan on every pull request — GitOps in practice
3. Read Netflix Tech Blog at netflixtechblog.com → find one post about their cloud-native architecture

✓ **QUICK SUMMARY**

- **DevOps = method (HOW):** culture, automation, CI/CD, monitoring, shared ownership
- **Cloud = toolbox (WHERE):** elastic infrastructure, APIs, managed services, global reach
- **Together:** cloud amplifies every DevOps practice — IaC, CI/CD, containers, monitoring
- **Cloud-Native:** designed for cloud from day one — not just "moved to cloud"
- **Serverless:** functions without servers — pay per call, zero idle cost, zero management
- **GitOps:** Git = single source of truth for infrastructure AND application code

■ **MINI QUIZ — Test Your Understanding**

Q1. Which statement BEST describes the relationship between DevOps and Cloud?

a) They are the same thing — cloud IS DevOps

b) DevOps replaces cloud computing

c) DevOps is the methodology; Cloud is the infrastructure platform they work on

d) You cannot do DevOps without cloud

✓ Answer: c — DevOps = the HOW (culture, methodology, automation). Cloud = the WHERE (elastic, programmable infrastructure). Complementary, not synonymous

Q2. Netflix intentionally kills production servers to test resilience. This practice is called:

a) Chaos Engineering

b) A serious accident

c) Load testing

d) Blue-green deployment

✓ Answer: a — Chaos Engineering: deliberately injecting failures to verify that systems handle them gracefully — pioneered by Netflix (Chaos Monkey)

Q3. What is the key difference between "lift-and-shift" and "cloud-native"?

a) Lift-and-shift is always better

b) Cloud-native means using more expensive instances

c) Lift-and-shift moves existing apps unchanged; cloud-native rebuilds apps for cloud patterns

d) They are the same — both run on cloud

✓ Answer: c — Lift-and-shift = move old app to VM without changes (misses cloud benefits). Cloud-native = rebuilt with containers, microservices, auto-scaling, managed services

CHAPTER 13 ■ Intermediate ■ 10 min read 13/14

Real-World DevOps & Cloud Workflow

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ❖ Trace a code change from developer laptop to production through all 7 pipeline stages
- ❖ Name the specific tool used at each stage of a real DevOps pipeline
- ❖ Explain what happens when a deployment fails and why automated rollback matters
- ❖ Describe what a DevOps engineer actually does in a typical workday

■ ANALOGY

DevOps pipeline = airport check-in → security → boarding → flight → landing → baggage → you → automated at every step

◆ DID YOU KNOW? ◆

<10 min	0	Auto
full pipeline: commit to production	manual server touches in mature pipeline	rollback on error spike

The 7-Stage Real-World Pipeline — Step by Step

- **Stage 1 — Developer Laptop:** local development and commit
  - → → engineer writes code in VS Code, runs unit tests locally
  - → → commits to a feature branch: git commit -m "feat: add cheque scanner"
  - → → opens a Pull Request on GitHub for peer code review
  - → → teammates review code, leave comments, approve the PR
- **Stage 2 — Git Repository:** merge triggers the pipeline
  - → → approved PR merges to main branch
  - → → GitHub/GitLab webhook fires → notifies CI server of new commit
  - → → entire pipeline starts automatically — no human action needed
- **Stage 3 — CI Server:** build and automated testing
  - → → GitHub Actions / Jenkins pulls the latest code
  - → → runs: build (docker build), unit tests, integration tests, security scan
  - → → if any test fails → pipeline stops → developer notified immediately
  - → → if all pass → build artefact (Docker image) created with SHA tag
- **Stage 4 — Artifact Registry:** store the deployable image
  - → → Docker image pushed to: AWS ECR / Docker Hub / GitHub Container Registry
  - → → image tagged with: Git commit SHA (e.g., myapp:a3f89b1)
  - → → image is immutable — same image deployed to staging AND production
  - → → audit trail: every image traceable to an exact commit and CI run

- **Stage 5 — Staging Environment:** pre-production validation
  - → → ArgoCD / Helm deploys the image to a cloud staging Kubernetes cluster
  - → → integration tests, smoke tests, and performance tests run automatically
  - → → staging is identical to production (built from same IaC)
  - → → if staging tests fail → pipeline stops, team notified, production untouched
- **Stage 6 — Production Cloud:** deployment to live users
  - → → Continuous Delivery: human clicks "approve" in Slack or GitHub
  - → → Continuous Deployment: promotion happens automatically if staging passes
  - → → deployment strategy: rolling update / blue-green / canary (1% then 100%)
  - → → if error rate spikes → automatic rollback to previous image within 2 minutes
- **Stage 7 — Monitoring & Alerting:** watch, learn, improve
  - → → Prometheus scrapes metrics every 15 seconds from all production pods
  - → → Grafana dashboards display: error rate, latency P99, throughput, CPU/memory
  - → → PagerDuty: alerts on-call engineer if error rate > 1% or latency > 500ms
  - → → insights from monitoring feed back into the next Plan stage

Stage	Tool Examples	Pass Condition	Fail Action
1. Developer	Git, VS Code, GitHub	PR approved by 2 reviewers	Fix review comments
2. Git Repo	GitHub, GitLab, Bitbucket	Merge to main	Resolve merge conflicts
3. CI Server	GitHub Actions, Jenkins	All tests pass, security clean	Notify dev, block pipeline
4. Artifact Reg	AWS ECR, Docker Hub	Image pushed successfully	Retry build
5. Staging	ArgoCD, Helm, Kubernetes	Integration tests pass	Notify team, block prod deploy
6. Production	ArgoCD, AWS ECS	Error rate < 1%	Auto-rollback to previous
7. Monitoring	Prometheus, Grafana, PagerDuty	SLA metrics within thresholds	Page on-call engineer

■ REAL-WORLD EXAMPLE

GitHub (the platform itself)

Situation:

- Ships features to 100M+ developers using its own CI/CD platform (GitHub Actions)

What they did:

- Every code change triggers a GitHub Actions pipeline — pipeline runs in under 5 minutes
- Deploys use feature flags — new features ship to 1% of users first, then gradually 100%
- Automated rollback triggers if error rate exceeds threshold within 10 minutes of deploy

Result:

- Hundreds of production deployments per week — no deployment fear, no "deploy freeze"
- P99 latency maintained under 200ms even during major feature launches

■ TRY IT YOURSELF

1. Fork any GitHub repo → enable GitHub Actions → make a small change → watch the full CI pipeline execute
2. Set up a simple Prometheus + Grafana stack locally using Docker Compose → create a dashboard with one metric
3. Read the GitHub Engineering Blog (github.blog/engineering) → find one post about their deployment pipeline

✓ QUICK SUMMARY

- **7 stages:** Laptop → Git → CI → Registry → Staging → Production → Monitoring
- **Full pipeline time:** commit to production in under 10 minutes in mature teams
- **Zero manual touches:** no engineer SSHes into a server — all automated
- **Immutable artefacts:** same Docker image deployed to staging AND production — guaranteed consistency
- **Automatic rollback:** error rate spike → previous version restored in 2 minutes automatically
- **Monitoring closes the loop:** production insights drive the next planning cycle

■ MINI QUIZ — Test Your Understanding

Q1. Why should the same Docker image be deployed to both staging and production (not rebuilt)?

a) To save build time

b) Images are too large to rebuild

c) Same image guarantees staging tests validate exactly what runs in production

d) Docker does not support rebuilding images

✓ Answer: c — Immutable artefacts: build once, deploy everywhere. Rebuilding for production could introduce differences that staging did not test

Q2. A deployment causes error rates to spike from 0.1% to 3%. What should happen in a mature DevOps pipeline?

a) Page engineers to manually fix the code immediately

b) Automatic rollback to the previous working version

c) Take the site offline while engineers debug

d) Send an email to the development manager

✓ Answer: b — Automated rollback: monitoring detects the spike → pipeline automatically re-deploys the previous stable version → error rate recovers within minutes

Q3. What tool watches Git and automatically deploys changes to Kubernetes clusters (GitOps pattern)?

a) GitHub Actions

b) Docker Hub

c) ArgoCD or Flux

d) Prometheus

✓ Answer: c — ArgoCD and Flux implement GitOps: they continuously watch a Git repo and automatically apply any changes to the target Kubernetes cluster

CHAPTER 14 ■ Beginner ■ 9 min read 14/14

Career Path Overview

■ LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✦ Map the 5-step DevOps learning staircase in order with time estimates
- ✦ Identify which DevOps role matches your current background and experience
- ✦ Name 3 certifications that increase hiring chances for DevOps roles
- ✦ Describe what a Junior DevOps Engineer does on a typical first week

■ ANALOGY

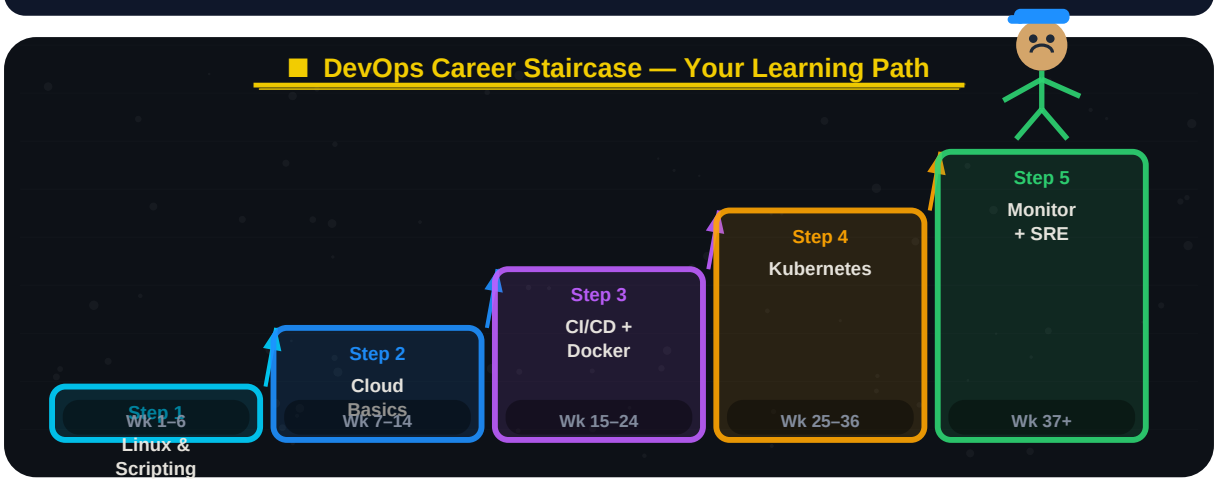
DevOps career = staircase not elevator → each step builds skills for the next, no shortcuts required

◆ DID YOU KNOW? ◆

Step 1
Linux & Scripting the foundation of all

Step 5
Security & SRE the advanced level

6 mo
from zero to Junior DevOps role
★ YOU!



The 5-step DevOps career staircase — build each skill level before moving to the next

The 5-Step Learning Staircase

- **Step 1 — Linux & Scripting (Weeks 1–6)**
  - → → **why first:** every production server, container, and cloud VM runs Linux
  - → → **learn:** terminal commands, file permissions, processes, bash scripting, networking basics
  - → → **goal:** be comfortable in a terminal — write scripts to automate repetitive tasks
  - → → **free resources:** Linux Journey (linuxjourney.com), "The Linux Command Line" (free PDF)
  - → → **milestone:** write a bash script that monitors disk usage and sends a Slack alert
- **Step 2 — Cloud Basics — AWS (Weeks 7–14)**
  - → → **why AWS:** 32% market share → highest job demand → most learning resources
  - → → **learn:** EC2, S3, IAM, VPC, RDS, Route53, AWS CLI, basic networking

→ → **goal:** confidently provision and manage cloud resources using CLI and console

→ → **certification:** AWS Cloud Practitioner (CCP) — $100, validates cloud fundamentals

→ → **free resources:** AWS free tier, AWS Skill Builder, A Cloud Guru free tier

- **Step 3 — CI/CD & Docker (Weeks 15–24)**
  → → **why this next:** CI/CD is the daily work of most DevOps roles
  → → **learn:** Docker (build images, run containers), GitHub Actions, Jenkins pipelines
  → → **goal:** build a CI/CD pipeline that builds, tests, and deploys a containerised app
  → → **portfolio project:** Python Flask app → Dockerised → CI/CD to AWS ECS Fargate
  → → **free resources:** Docker official docs, GitHub Actions quickstart, TechWorld with Nana (YouTube)
- **Step 4 — Kubernetes (Weeks 25–36)**
  → → **why:** K8s is the production standard for container orchestration worldwide
  → → **learn:** Pods, Deployments, Services, ConfigMaps, Namespaces, Helm, kubectl
  → → **goal:** deploy and scale a multi-service application on a Kubernetes cluster
  → → **certification:** CKA (Certified Kubernetes Administrator) — highly valued by employers
  → → **free resources:** Kubernetes.io docs, Nana's K8s series (YouTube), killercoda.com labs
- **Step 5 — Monitoring, Security & SRE (Weeks 37+)**
  → → **learn:** Prometheus + Grafana, secrets management (Vault), container scanning (Trivy)
  → → **SRE concepts:** error budgets, SLOs, SLIs, incident management, chaos engineering
  → → **DevSecOps:** shift security left — SAST, DAST, dependency scanning in CI pipelines
  → → **free resources:** Google SRE Book (sre.google/sre-book), HashiCorp Vault docs, OWASP
  → → **advanced certification:** Google Professional DevOps Engineer, AWS DevOps Engineer Professional

Role	Core Skills Required	Experience	Salary (India)	Salary (Global)
Junior DevOps	Linux, Git, basic CI/CD, one cloud provider	0–2 yrs	■5–10 LPA	$70–95K USD
DevOps Engineer	IaC, Kubernetes, CI/CD, monitoring, scripting	2–4 yrs	■12–22 LPA	$100–130K USD
Senior DevOps / SRE	Architecture, security, reliability engineering	4–7 yrs	■22–40 LPA	$140–170K USD
Platform Engineer	Internal dev platforms, golden paths, APIs	5+ yrs	■30–50 LPA	$160–190K USD
Cloud Architect	Multi-cloud strategy, enterprise design	8–12 yrs	■50+ LPA	$180–220K USD

Key Certifications — Prioritised

- **AWS Cloud Practitioner (CCP):** start here — validates cloud fundamentals → $100 → online, 90 min
- **HashiCorp Terraform Associate:** second cert — IaC skills are highly valued → $70.50 → online

- **AWS Solutions Architect Associate (SAA-C03):** most popular cloud cert → $150 → highly respected
- **CKA (Certified Kubernetes Administrator):** hands-on Kubernetes exam → $395 → industry standard
- **AWS DevOps Engineer Professional:** advanced — requires SAA first → $300 → senior roles
- **Google Professional DevOps Engineer:** alternative to AWS path → $200 → valued at Google-heavy shops

■ REAL-WORLD EXAMPLE

Real Career Transition

Situation:

• Windows System Administrator managing 50 on-premise servers — felt stuck, underpaid

What they did:

• Spent 6 months: learned Linux (2mo), AWS CCP cert (1mo), built Docker CI/CD pipeline (2mo), studied K8s (1mo)

• Built a public GitHub portfolio: Terraform modules, GitHub Actions pipeline, Dockerised app

• Applied for Junior DevOps roles with 2 years of sysadmin experience + 6 months self-study

Result:

• Hired as Junior DevOps Engineer at a fintech startup

• Salary doubled: from ■6 LPA → ■13 LPA in first year

• Within 3 years: promoted to Senior DevOps Engineer at ■28 LPA

■ COMMON MISCONCEPTIONS

■ **Myth:**

"I need to know everything before I apply for Junior DevOps roles"

■ **Reality:**

Apply after completing Steps 1-3. Employers hire for: fundamentals + learning ability + problem-solving attitude. You learn the rest on the job — that's what Junior means

■ **Myth:**

"DevOps is only for experienced engineers with CS degrees"

■ **Reality:**

Many successful DevOps engineers: came from sysadmin, QA, support, or non-CS backgrounds. The community is welcoming, resources are free, and skills are learnable by anyone willing to practice daily

■ TRY IT YOURSELF

1. TODAY: Create a GitHub account → create a repository → commit a README.md that describes your DevOps learning goals
2. THIS WEEK: Install Linux (WSL2 on Windows or Ubuntu VM) → complete linuxjourney.com first 2 modules
3. THIS MONTH: Create your free AWS account → complete the AWS Cloud Practitioner Essentials course on AWS Skill Builder (free)

✓ **QUICK SUMMARY**

- **5 steps:** Linux → Cloud → CI/CD+Docker → Kubernetes → Monitoring+Security
- **Start now:** open a terminal, type ls -la — your DevOps journey begins with that first command
- **Portfolio > certifications:** working GitHub projects get interviews; certs alone do not
- **Apply at Step 3:** do not wait for perfection — employers hire attitude and fundamentals
- **Community:** r/devops, CNCF Slack, local Meetup.com events — welcoming and resourceful
- **Timeline:** 6 months of consistent daily practice → ready for Junior DevOps roles

■ **MINI QUIZ — Test Your Understanding**

Q1. What should a complete beginner learn FIRST on the DevOps career path?

a) Kubernetes — it is the most in-demand skill

b) Linux and Scripting — it is the foundation for every other tool

c) AWS certifications — employers require them

d) Docker — containers are the future

✓ Answer: b — Linux is the foundation: every server, container, Kubernetes node, and CI/CD runner runs Linux. You cannot automate what you cannot navigate manually

Q2. A junior DevOps applicant has no certifications but has a GitHub portfolio with a working CI/CD pipeline, Terraform module, and Dockerised app. Will they get interviews?

a) No — certifications are required for all DevOps roles

b) Yes — working code demonstrating practical skills is often more valuable than certifications

c) Only if they have a CS degree

d) Only for senior roles — juniors need no portfolio

✓ Answer: b — Portfolio evidence of practical skills (working pipelines, IaC code, deployed apps) is highly valued by DevOps hiring managers — often more than paper certifications

Q3. Which certification is the best STARTING POINT for a DevOps career?

a) CKA (Certified Kubernetes Administrator)

b) AWS DevOps Engineer Professional

c) AWS Cloud Practitioner (CCP)

d) Google Professional Data Engineer

✓ Answer: c — AWS CCP is entry-level, vendor-neutral in cloud concepts, low cost ($100), and validates you understand cloud fundamentals — ideal first certification before SAA or CKA

Glossary of Key Terms — Day 01

Ansible	Agentless configuration management tool — uses SSH + YAML playbooks to configure servers
ArgoCD	GitOps CD tool — watches Git repos and auto-deploys changes to Kubernetes clusters
Artefact	Output of a successful build: Docker image, JAR file, or binary — versioned and immutable
Auto Scaling	Cloud feature that automatically adds/removes resources based on demand metrics
Availability Zone	Physically separate data centre within a cloud region — independent power and networking
Blameless Post-Mortem	Incident review focused on system improvement — not individual blame
C.A.L.M.S.	Culture, Automation, Lean, Measurement, Sharing — the 5 pillars of DevOps culture
CapEx	Capital Expenditure — upfront cost of buying hardware (traditional IT model)
CD (Continuous Delivery)	Every passing build is deployment-ready; human approves the final production push
CD (Continuous Deployment)	Every passing build auto-deploys to production — zero human intervention
CI (Continuous Integration)	Merge code frequently; auto build and test on every push to catch issues early
Cloud-Native	Applications designed from day one for cloud: containers, microservices, managed services
CKA	Certified Kubernetes Administrator — hands-on exam, highly valued by employers
Chaos Engineering	Intentionally inject failures into production to verify system resilience (Netflix: Chaos Monkey)
DORA Metrics	4 key DevOps metrics: deployment frequency, lead time, MTTR, change failure rate
Dev (Development)	Team responsible for writing application code, designing features, and fixing bugs
DevOps	Culture + methodology combining Dev and Ops with shared ownership, automation, and CI/CD
ElasticSearch	Search engine (often part of ELK stack for log analysis in monitoring)
Elasticity	Cloud ability to automatically scale resources up or down based on demand
Error Budget	Tolerable amount of downtime/errors within an SLO period — consumed by risky changes
Fail Fast	Detect problems at the earliest pipeline stage (cheap) rather than in production (expensive)

FinOps	Cloud financial management: rightsizing, budget alerts, reserved instances, cost attribution
FaaS	Function as a Service — run individual functions without managing servers (AWS Lambda)
GitOps	Git as single source of truth for both app code AND infrastructure state (ArgoCD, Flux)
Helm	Kubernetes package manager — templates and manages complex application deployments
IaC (Infrastructure as Code)	Managing infrastructure through version-controlled code files, not manual steps
IaaS	Infrastructure as a Service — rent raw VMs; you manage OS and everything above
Idempotent	Running the same operation multiple times always produces the same result — no duplicates
Integration Hell	Painful process of merging months of separate development branches all at once
Kubernetes (K8s)	Container orchestration platform — automates deployment, scaling, and management of containers
Lead Time	Time from code commit to production deployment — DORA metric (elite: under 1 hour)
MTTR	Mean Time to Recovery — average time to restore service after failure (elite: under 1 hour)
Microservices	Architecture where an app is many small, independent, separately deployable services
Multi-Cloud	Using multiple cloud providers simultaneously: AWS + GCP + Azure
Observability	Understanding system state from outputs: metrics, logs, and traces
On-Demand	Cloud characteristic: get resources instantly without procurement or waiting
OpEx	Operating Expenditure — pay-as-you-go model (cloud computing cost model)
Ops (Operations)	Team managing servers, cloud, networking, security, and 24/7 uptime
PaaS	Platform as a Service — deploy code without managing infrastructure (Heroku, App Engine)
Pipeline	Ordered sequence of automated stages: Commit → Build → Test → Deploy → Monitor
Prometheus	Open-source metrics collection system — de-facto standard for cloud-native monitoring
Rollback	Automatic reversal to previous working version when error rates spike after deployment
SaaS	Software as a Service — use finished software over internet; manage only your data (Gmail, Slack)
SLA	Service Level Agreement — formal uptime contract (e.g., 99.9% = max 8.7 hours downtime/year)
SLI	Service Level Indicator — the actual measured metric (e.g., measured uptime percentage)

SLO	Service Level Objective — internal target for reliability (e.g., 99.9% uptime over 30 days)
SRE	Site Reliability Engineering — applying software engineering practices to operations and reliability
Serverless	Run code without managing servers — pay per invocation, zero idle cost (AWS Lambda)
Silos	Teams working in complete isolation, hoarding knowledge instead of sharing
Terraform	HashiCorp IaC tool — declaratively provisions cloud infrastructure across any provider
Toil	Manual, repetitive, operational work that scales linearly — should be automated
Wall of Confusion	Cultural/technical barrier between Dev and Ops — root cause of most traditional IT failures
Waterfall	Sequential development model: Requirements→Design→Code→Test→Deploy — one phase at a time

Resources & Further Learning — Day 01

■ Essential Books

- "The Phoenix Project" by Gene Kim — the DevOps novel (read this first)
- "The DevOps Handbook" by Gene Kim et al. — comprehensive practices guide
- "Site Reliability Engineering" by Google — free at sre.google/sre-book
- "Accelerate" by Nicole Forsgren — data science behind DevOps (DORA research)

■ Learning Platforms

- KodeKloud (kodekloud.com) — best hands-on DevOps labs with browser terminals
- A Cloud Guru (acloudguru.com) — AWS, Azure, GCP, Kubernetes courses
- Linux Foundation Training — official Linux, Kubernetes, and cloud-native courses
- AWS Skill Builder (skillbuilder.aws) — free AWS courses including CCP prep

■ YouTube Channels

- TechWorld with Nana — best DevOps YouTube for beginners (highly recommended)
- NetworkChuck — fun Linux, networking, and cloud content with great energy
- Fireship — "X in 100 Seconds" series for quick concept overviews
- AWS and Google Cloud official channels — free certification prep content

■ Free Hands-On Practice

- Killercoda (killercoda.com) — browser-based Linux and Kubernetes labs, no install
- Play with Docker (labs.play-with-docker.com) — free Docker playground in browser
- AWS free tier (aws.amazon.com/free) — 12 months of free cloud experimentation
- GitHub (github.com) — free repos, CI/CD Actions, container registry, Codespaces

■ Certifications — Recommended Order

- Start: AWS Cloud Practitioner (CCP) — $100, validates cloud fundamentals
- Next: HashiCorp Terraform Associate — $70.50, IaC skills highly valued
- Then: AWS Solutions Architect Associate (SAA-C03) — $150, most popular cert
- Advanced: CKA (Certified Kubernetes Administrator) — $395, hands-on exam

■ Community & GitHub Repo

- github.com/aryashreep/learn-devops-in-90-days — companion repo for this series
- DevOps Subreddit (reddit.com/r/devops) — questions, career advice, tool discussions
- CNCF Slack (slack.cncf.io) — cloud-native community, K8s, Prometheus, ArgoCD channels
- Meetup.com — search "DevOps" or "Cloud" + your city for local events and networking

Begin Today — Open a Terminal and Type: ls -la

Day 01 Complete ✓ • Next: Day 02 will build on today's foundations

github.com/aryashreep/learn-devops-in-90-days • DevOps 90-Day Training 2026 • Aryashree Pritikrishna